OPERATING SYSTEMS

ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks

You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation

You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating
- Explain position-independent code

Experimental Investigation

You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks

You must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

Evaluation

You must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation

You must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
        //   PAGE_SIZE. If p is outside the table print "invalid address"; if the frame
        //   is -1 print "page fault"; otherwise print the physical address frame *
        //   PAGE_SIZE + d

    }
    return 0;
}
```

Expected output:

```
   0 -> page 0 offset    0 : physical 5120
1500 -> page 1 offset  476 : physical 2524
3100 -> page 3 offset   28 : physical 7196
6200 -> page 6 offset   56 : physical 3128
9000 -> page 8 offset  808 : invalid address
2049 -> page 2 offset    1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks

You must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation

You must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation

You must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes
- Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks

You must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation

You must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation

You must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks

You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation

You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it
- Compare the algorithms for a workload with strong locality

Experimental Investigation

You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

> *Which page-replacement policy would you recommend for the web server? Use your measurements.*

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```c
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        // TODO 1: If a frame is free use it; otherwise replace the frame whose last use
        //   (last[]) is the oldest. Record last[] = i for the frame used

    }
    return faults;
}

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len;                  /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
               lru(ref, len, fr), optimal(ref, len, fr));
    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

Expected output:

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management **4 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks

You must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation

You must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation

You must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock())
- Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator
- Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ASSIGNMENT 3 (CO3) — ANSWER KEY / SOLUTION NOTES

These notes answer every question of Assignment 3 (linking/loading, paging, segmentation, virtual memory, demand paging, page-replacement and disk management). They are written as a model solution to guide the student's own report, code execution, GitHub submission and viva preparation. Each student must still execute every program on their own Linux machine, capture real screenshots, ldd/strace/time/getconf output and write the README.md as instructed on the cover pages.

Q1 – Linking, Loading and Address Binding (6 Marks)

Implementation: building the same banking program twice — once with gcc prog.c -static -o prog_static (static linking, pulling the whole C library and the encryption library into one executable) and once with gcc prog.c -lcrypto -o prog_dynamic (dynamic/shared linking against libcrypto.so) — and comparing ls -l prog_static prog_dynamic shows the static binary is dramatically larger (it embeds the library code itself) while the dynamic binary is small (it only stores a reference to the shared object). ldd prog_dynamic lists every shared library the dynamic executable depends on (e.g., libcrypto.so.3, libc.so.6, the dynamic linker/loader itself ld-linux.so.2) together with the path each is resolved to at run time. dlopen("libcrypto.so.3", RTLD_LAZY) followed by dlsym(handle, "SHA256") loads a library and resolves one of its symbols at run time, on demand, rather than at program start. /proc/self/maps, read from inside the running program (or /proc/<pid>/maps from outside), lists every memory region mapped into the process, including the base address each shared library and the main executable itself were loaded at — on a system with ASLR enabled these addresses differ between runs.

Evaluation — address binding: compile-time binding fixes absolute memory addresses into the executable at compile time (only possible if the exact load address is known in advance — rare today); load-time binding lets the compiler generate relocatable code and the loader fix in the actual addresses when the program is loaded into memory (used when the load address may vary but is fixed once loading starts); execution-time (dynamic) binding defers the final address resolution until run time and allows a process to even move in memory during execution (needed for shared libraries, position-independent code and ASLR). The linker combines separately compiled object modules and resolves symbol references between them into a single executable (or into a shared library); the loader brings that executable into memory, allocates its address space, maps its shared library dependencies and transfers control to its entry point.

Evaluation — static vs dynamic linking: static linking uses more disk/memory per executable (each program embeds a full private copy of every library it uses), has slightly faster start-up (no run-time symbol resolution/relocation needed) but is hard to update (a library security fix — such as the bank's encryption fix — requires rebuilding and redistributing every program that uses it, exactly the 200 MB-per-branch problem in the scenario). Dynamic linking uses less memory overall (one copy of a shared library's code pages can be mapped read-only and shared by every process using it, confirmed by comparing /proc/<pid>/maps of two running copies, which should show the same physical shared-library pages mapped into both), has slightly slower start-up (the loader must resolve symbols and apply relocations at load time) but updating the shared library file alone instantly fixes every program that loads it next time, without recompiling any of them — directly solving the bank's stated problem.

Position-independent code (PIC): code compiled so that all internal references (jumps, data accesses) are relative to the current execution location rather than to a fixed absolute address, typically using a Global Offset Table (GOT) and Procedure Linkage Table (PLT) for external symbols — this lets the same compiled shared-library code be mapped at a different address in every process (and at a different, randomised address on every run, for ASLR) without needing to be re-linked or patched per process.

Experimental investigation: start-up time and memory use are measured with time ./prog and ps/ /proc/<pid>/status (VmRSS); replacing the shared library (installing a new libcrypto.so.3 in place) and re-running the dynamic binary without recompiling it demonstrates the update taking effect immediately (the statically linked binary, by contrast, keeps using its old embedded copy until rebuilt); running two copies of the

dynamic binary simultaneously and inspecting /proc/<pid>/maps for both shows the same shared-library file mapped at (possibly different, due to ASLR, but logically the same) addresses, backed by the same physical pages, confirming sharing; removing or renaming the shared library and re-running the dynamic binary produces a run-time "error while loading shared libraries" failure (since resolution is deferred to execution time), whereas the static binary is unaffected because it has no such external dependency.

Conclusion: dynamic (shared) linking is the right choice for the bank, because the deciding factor stated in the scenario — being able to push a security fix to every branch without rebuilding and redistributing a 200 MB executable — is exactly what dynamic linking enables; the extra per-process memory for private data segments and the small run-time symbol-resolution overhead are an acceptable cost for that operational benefit.

Alternative Concept and Complexity Analysis — loading the encryption library with dlopen() only when needed, instead of linking it dynamically at start-up: start-up time improves for branch-software sessions that never use encryption in a given run, since the library's code pages and relocation work are deferred until the first dlopen() call rather than paid unconditionally at process start; for sessions that do use encryption, the one-time dlopen() cost is paid later instead of earlier, so total work done is similar, only its timing shifts. Memory used when encryption is not needed falls, because the library's pages are never mapped at all in that run (versus ordinary dynamic linking, which maps the library at start- up whether or not it is ever actually called). Error handling becomes the programmer's explicit responsibility: a missing library now produces a NULL return from dlopen() that the program must check and handle gracefully (e.g., disabling the encryption feature with a clear message), rather than refusing to start at all as an ordinary dynamically-linked program would — this trades an early, obvious failure for a later, code-path-dependent one that must be deliberately tested.

Q2 – Paging (8 Marks)

External fragmentation under contiguous allocation: as programs of different sizes are loaded and removed from a single contiguous address space, the free memory left behind is broken into many small, scattered holes; even when the total free memory is more than enough for a new, large program, no single hole may be big enough to hold it — this is exactly the college server's stated problem ("large programs could not be loaded although enough total memory was free").

Address translation with a page table: a logical address is split into a page number p (selects an entry in the page table) and an offset d (position within the page); the page table entry for p gives the physical frame number f; the physical address is then f × (page size) + d. This is illustrated directly by Q2's completed TODO-1 program below.

Page-table size: for a 32-bit address space with 4 KB (2^12-byte) pages, there are 2^32 / 2^12 = 2^20 ≈ 1 million page-table entries per process; at a typical 4 bytes per entry that is about 4 MB of page table for one process's full address space (hence multi-level page tables are used in practice, so that unused regions of a sparse address space need no entries at all). For a 48-bit address space with 4 KB pages there are 2^48 / 2^12 = 2^36 ≈ 68.7 billion entries, which at 8 bytes per entry would need roughly 549 GB of page table if implemented as one flat table — this huge number is exactly why real 48-bit systems always use a hierarchical (multi-level) page table, where only the branches of the tree that are actually in use consume memory.

Effective memory-access time (EMAT): without a TLB, every memory reference costs two real memory accesses (one to read the page table entry, one to read/write the actual data), so EMAT = 2 × memory-access-time. With a TLB of hit ratio h and TLB-access time t (much smaller than memory time m), EMAT = h × (t + m) + (1 – h) × (t + m + m) = t + m + (1 – h) × m — i.e. a TLB turns most accesses into roughly one memory access plus a fast TLB look-up, instead of two full memory accesses, which is the entire reason TLBs exist.

Evaluation: paging removes external fragmentation because every page is the same fixed size and can be placed in any free frame, so free memory is never broken into unusable odd-sized holes — but internal fragmentation remains, because a process's last page is usually only partially used, wasting up to (page size –

1) bytes per process on average half a page. Page-size trade-off: a small page (1 KB) gives low internal fragmentation but a very large page table (many entries) and more TLB misses for a given working set (since the TLB covers less address space per entry); a large page (2 MB, "huge pages") gives a tiny page table and far fewer TLB misses, but high internal fragmentation for small processes/files and coarser-grained protection; 4 KB is the conventional middle ground balancing both costs for a general- purpose workload such as the college server's. Hierarchical (multi-level) page tables save memory for sparse address spaces by only allocating inner-level tables for ranges that are actually mapped; hashed page tables map a (process-id, page-number) pair through a hash function into a chained table, used for very large (e.g., 64-bit) address spaces where a flat or even multi-level table would still be too large; inverted page tables keep one entry per physical frame (not per virtual page), so their size is proportional to physical memory rather than to the (possibly much larger) address space of every process combined, at the cost of a slower, often hashed, look-up since the table is no longer indexed directly by virtual page number. Protection is added per page-table entry (read/write/execute/user-kernel bits checked by the MMU on every access); sharing is achieved by mapping the same physical frame into the page tables of two or more processes (e.g., for a shared library's code pages), each usually marked read-only to protect it from being modified by any one process.

Experimental investigation: internal fragmentation for 100 random-sized processes is computed as the sum, over all processes, of (page size − (process size mod page size)) when the remainder is non-zero, and this total should be shown to shrink roughly proportionally as page size shrinks (1 KB vs 4 KB vs 2 MB). Effective access time at TLB hit ratios of 80%, 95%, 99% should be computed with the EMAT formula above and should show markedly better performance as the hit ratio rises, with diminishing improvement near 99%+ (since even a hit ratio of 95% already removes most of the extra memory access, and the law of diminishing returns applies as h→1). getconf PAGESIZE reports the student's own Linux page size (typically 4096 on x86-64); /usr/bin/time -v ./program reports "Minor (reclaiming a frame) page faults" and "Major (requiring I/O) page faults" for a chosen test program.

Conclusion: 4 KB remains the recommended page size for the general-purpose college server, because it keeps the page-table size and internal fragmentation at a manageable, well-understood level for the server's mixed workload of many differently-sized student programs; 2 MB huge pages could be offered selectively for a few known large, memory-intensive jobs (e.g., simulations) to cut their TLB-miss rate, but are a poor default for the general student population whose programs are typically small.

```
#include <stdio.h>
#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];

        /* TODO 1 solution */
        unsigned p = a / PAGE_SIZE;
        unsigned d = a % PAGE_SIZE;

        if (p >= PAGES) {
            printf("%u -> page %u offset %u : invalid address\n", a, p, d);
        } else if (page_table[p] == -1) {
            printf("%u -> page %u offset %u : page fault\n", a, p, d);
        } else {
            unsigned phys = page_table[p] * PAGE_SIZE + d;
            printf("%u -> page %u offset %u : physical %u\n", a, p, d, phys);
        }
    }
    return 0;
}
```

With PAGE_SIZE changed to 2048, page numbers and offsets must be recalculated by hand before re-running: e.g., 1500 → page 0, offset 1500 (since 1500 < 2048); 3100 → page 1, offset 1052; 6200 → page 3, offset 56; 9000 → page 4, offset 904 (now within range since PAGES=8 covers 8×2048=16384 bytes, so

"invalid address" at 9000 no longer occurs); 2049 → page 1, offset 1 (page_table[1] = 2, so it now resolves to a valid physical address rather than a page fault) — the student should verify these by hand and then confirm the program's actual output matches.

Alternative Concept and Complexity Analysis — an inverted page table instead of a per-process multi-level page table: space used by an inverted table is proportional to the number of physical frames in the whole machine (one entry per frame, shared across all processes), which for a fixed amount of physical RAM stays constant no matter how many processes run or how large each one's virtual address space is — in contrast, a per-process multi-level table's total size grows with the number of processes and the portion of each one's address space that is actually mapped. Translation time for an inverted table requires searching (typically via a hash table keyed by (process-id, virtual page number)) for the frame that currently holds that page, which is usually O(1) average case with a good hash function but can degrade under collisions, versus a multi-level table's translation, which is a fixed number of sequential memory reads (one per level, e.g. 2–5 for typical multi-level schemes) — both are normally backed by a TLB in practice so the raw table-walk cost is rarely paid on most accesses.

Q3 – Segmentation (6 Marks)

Address translation with a segment table: a logical address is given as (segment number s, offset d); the segment table entry for s stores a base (the segment's starting physical address) and a limit (the segment's length); if d < limit, the physical address is base + d; if d ≥ limit, the hardware raises a segmentation fault (an out-of-bounds trap to the OS), which is precisely what happens when a program accesses memory beyond an array's declared segment — the real-world "segmentation fault" error.

Segmentation with paging (as in x86): the logical address is first translated through a segment table (giving a linear/virtual address within that segment, bounded by the segment's limit), and that linear address is then itself paged (split into page number and offset, translated through a page table into a physical frame) — combining segmentation's logical, variable-sized, protectable units (code, stack, data) with paging's fixed-size, fragmentation-free physical-memory allocation.

/proc/<pid>/maps on Linux shows the logical segments of a running program as a list of mapped regions, each with its address range, permission bits (r/w/x), and what it is backing (the executable's text segment, its data/BSS segment, the heap, the stack, and each shared library mapped in) — a practical, OS-level view of segmentation-like structure even though Linux itself uses paging as its primary mechanism and only a minimal flat segmentation model at the hardware level.

Evaluation: segmentation matches the programmer's logical view of a program (separate code, stack, data, and perhaps per-function or per-module segments) far more directly than paging's arbitrary, fixed-size slices, which makes protection and sharing natural at a meaningful granularity — e.g., the whole code segment can be marked read-only and shared by every process running the same compiler/program, exactly as the scenario requires; a growing stack segment can simply have its limit extended, without needing contiguous physical memory, since its base/limit only govern the logical-to-physical mapping. Protection is applied per segment (read/write/execute permission bits in the segment-table entry, checked on every access); sharing is achieved by pointing two processes' segment-table entries at the same physical base (e.g., sharing one compiler's code segment among every process running it). External fragmentation returns with pure segmentation, because segments are variable-sized, so (just like the original contiguous- allocation problem in Q2) loading and removing segments of different sizes over time leaves scattered, possibly unusable holes in physical memory. Pure paging has no external fragmentation (fixed-size frames) but only internal fragmentation and no natural match to the programmer's logical units; pure segmentation has a natural logical match and no internal fragmentation (a segment is exactly as big as it needs to be) but reintroduces external fragmentation; segmented paging (Intel x86) gets the logical/protection benefits of segmentation plus the fragmentation-free physical allocation of paging, at the cost of two levels of translation table and hardware complexity.

Experimental investigation: translating 8 hand-chosen (segment, offset) addresses against a small segment table (verified both by hand and with a short C program that mirrors Q2's style) should match; deliberately reading past an array's bound (or dereferencing a wild pointer) should reliably trigger a real Unix SIGSEGV, demonstrating the OS trapping an out-of-limit access and terminating the offending process rather than letting it corrupt unrelated memory; loading and removing 20 random-sized logical segments in a simulation and measuring the resulting total unusable gap size demonstrates segmentation's external fragmentation growing with the number of allocate/free cycles, similar in nature (though not identical in magnitude) to the original contiguous-allocation problem in Q2; comparing table sizes shows a segment table is typically much smaller (one entry per logical segment, perhaps a handful per process) than a page table (one entry per fixed-size page, potentially thousands per process).

Conclusion: modern operating systems rely mainly on paging (often with limited segmentation layered on top, as in x86's flat-segment-plus-paging model) because paging's fixed-size units eliminate external fragmentation entirely and give simple, uniform physical-memory management, while most of segmentation's useful protection/sharing properties can be reproduced at the page level (marking ranges of pages read-only or shared) without paying variable-size allocation's fragmentation cost — segmentation's chief remaining advantage, matching the programmer's logical view, is instead provided today by the virtual-memory layout the OS and linker construct (separate code/data/heap/stack regions) on top of an underlying, purely paged physical-memory system.

Alternative Concept and Complexity Analysis — pure paging instead of segmentation: fragmentation shifts from external (segmentation's variable-sized holes) to purely internal (paging's up-to-almost-one- page waste per allocation, as established in Q2) — generally a smaller, more predictable, and easier-to- manage total waste than the scattered external fragmentation segmentation can produce under heavy allocate/free churn. Table size falls, since pure paging needs only one flat (or hierarchical) page table per process rather than both a segment table and, under segmented paging, a page table per segment. Address translation time is typically faster or comparable (one table walk, possibly multi-level, versus segmentation's base+limit check followed by a further page-table walk under segmented paging). Sharing and protection are still achievable under pure paging — by marking the specific pages that happen to hold shared code (e.g., a library) as shared/read-only — but this requires that a logical unit (like "the code segment") already begins and ends on page boundaries to share or protect it cleanly as a whole, which is less natural than segmentation's explicit, meaningfully-sized units, and is the one genuine advantage pure paging gives up by abandoning segmentation.

Q4 – Virtual Memory and Demand Paging (8 Marks)

Steps in handling a page fault: (1) the CPU generates a memory reference to a page not currently marked present in the page table, trapping into the OS; (2) the OS checks whether the reference is valid (within the process's legitimate address space) — an invalid reference is treated as a protection violation and the process is terminated; (3) for a valid reference, the OS finds a free physical frame, or selects a victim frame to evict (via a page-replacement algorithm, Q5) if none is free; (4) if the victim page is dirty (modified), it is written back to backing storage; (5) the OS schedules a disk/SSD I/O operation to read the needed page into the chosen frame and blocks the faulting process while the I/O completes; (6) once the I/O finishes, the OS updates the page table to mark the page present and point to the new frame; (7) the faulting instruction is restarted from the beginning, this time succeeding because the page is now resident.

Effective access time (EAT) for page-fault rate p, memory access time 100 ns, fault-service time 8 ms (=8,000,000 ns): EAT = (1 – p) × 100 ns + p × 8,000,000 ns (approximately, ignoring the small extra memory access already folded into the 100 ns term for simplicity). For p = 1/1,000 = 0.001: EAT ≈ 0.999×100 + 0.001×8,000,000 ≈ 99.9 + 8,000 ≈ 8,099.9 ns. For p = 1/100,000 = 0.00001: EAT ≈ 100 + 80 ≈ 180 ns. For p = 1/1,000,000 = 0.000001: EAT ≈ 100 + 8 ≈ 108 ns. These figures show how dramatically a page-fault rate even as "small" as 1 in 1,000 inflates effective access time by roughly 80× versus the no-fault baseline, which is exactly why Q4's evaluation below caps the acceptable fault rate so tightly.

Copy-on-write and fork(): instead of physically duplicating the parent's entire address space at fork() time, the child's page table is set up to point at the very same physical frames as the parent's, with both marked read-only; only when either process actually writes to a shared page does the hardware trap (a protection fault), at which point the OS makes a private copy of just that one page for the writer and lets the write proceed — this makes fork() fast (O(page-table size), not O(address-space data size)) because the common case (an immediate exec() after fork(), as in Assignment 1 Task 7/2) never actually touches most of the duplicated pages at all.

Thrashing and the working-set model: thrashing occurs when a process (or the whole system) does not have enough frames to hold its working set — the set of pages it is actively referencing in its current phase of execution — so it continually faults pages in only to have them evicted again before they are reused, spending more time servicing page faults than doing useful work; system throughput can collapse even as CPU utilisation appears to drop (the classic signal that triggers the OS to reduce the degree of multiprogramming). The working-set model tracks, for each process, the set of pages referenced within the last Δ references (the working-set window); the OS aims to keep each process allocated at least as many frames as the size of its working set, and if the sum of all processes' working sets exceeds total available frames, the OS should suspend one or more processes rather than let everyone thrash.

Evaluation: demand paging lets a 24 GB program run in 16 GB of RAM by keeping only the pages currently being referenced resident in physical memory and fetching others from disk/SSD on demand, backed by swap space (or simply never loading unused code/data pages at all) — the program's full virtual address space can exceed physical RAM as long as its actual working set at any moment fits. The maximum acceptable page-fault rate for only a 10% slow-down is found by solving EAT $\leq 1.10 \times 100$ ns $= 110$ ns: $110 = (1-p)\times 100 + p\times 8{,}000{,}000 \approx 100 + p\times 7{,}999{,}900$, so $p \leq 10/7{,}999{,}900 \approx 0.00000125$, i.e. roughly 1 fault per 800,000 references — a strikingly small tolerance, showing how little room demand paging has before performance visibly degrades. Frame allocation can be equal (every process gets memory/number-of-processes frames, simple but ignores differing needs) or proportional (frames allocated in proportion to each process's size/working-set size, fairer for mixed workloads like the radiologist's single large imaging program sharing the laptop with smaller background tasks). The OS detects thrashing by monitoring the system-wide page-fault rate (and/or each process's working-set size versus its allocated frames); on detecting it, the OS reduces the degree of multiprogramming — suspending (swapping out entirely) one or more processes so the remaining ones each get enough frames for their working set, trading fewer concurrently-running processes for each one actually making progress instead of all of them thrashing.

Experimental investigation: a test program touching memory sequentially should show far fewer page faults than the same amount of memory touched in random order (sequential access benefits from spatial locality and OS read-ahead, so pages already brought in by prior faults are more likely to be reused before eviction, while random access defeats both locality and read-ahead); run time should rise sharply, non-linearly, once the memory the program touches exceeds available free RAM and the system must start paging to disk/swap, directly demonstrating thrashing; measuring memory (e.g., via /proc/<pid>/status VmRSS, or the shared "Pss" in /proc/<pid>/smaps) immediately after fork() and again after the child writes to its data shows RSS barely changing right after fork() (pages are still shared, copy-on-write) but shared/Pss dropping once the child's writes force private copies; plotting page faults against the number of frames available for a chosen reference string (as computed in Q5) should show faults falling as frames increase, flattening out once the process's full working set fits in the available frames.

Conclusion: for the radiologist, whether to buy more RAM (32 GB) or improve the program's locality depends on the measured fault rate and its EAT impact — if the measured page-fault rate at 16 GB is well above the $\approx$0.00000125 threshold derived above (likely, given the program needs 24 GB and RAM is 16 GB), the slow-down is severe, and buying more RAM (removing demand paging from the picture almost entirely for this workload) is the more reliable fix, since improving locality (e.g., processing the image in smaller tiles that better fit the frames available) helps but cannot by itself guarantee the working set shrinks enough to avoid expensive faults — a combination of both, prioritising the RAM upgrade if the measured slow-down is large, is the recommended, evidence-based answer.

Alternative Concept and Complexity Analysis — 32 GB of RAM instead of relying on demand paging at 16 GB: with the whole 24 GB program's working set fitting comfortably in 32 GB of physical RAM, the measured page-fault rate for ordinary operation should fall close to zero (faults limited mostly to the program's own cold-start loading and occasional background system activity), so effective access time approaches the bare 100 ns memory-access baseline rather than the thousands-of-ns EAT computed above at even a 1-in-1,000 fault rate — a very large, directly measurable improvement. The cost trade-off is the one-time price of the extra RAM (a fixed hardware cost) against the recurring cost of the slower run (radiologist time lost to a sluggish, faulting application every time the 24 GB program is used) — for a workload used routinely rather than occasionally, the RAM upgrade typically pays for itself quickly in recovered productivity, which is why it is the generally preferred conclusion above.

Q5 – Page-Replacement Techniques (8 Marks) — Complete the Code

FIFO replaces the page that has been resident the longest, regardless of how recently or frequently it was used — simple (just a queue) but can perform poorly, and uniquely among common algorithms can suffer Belady's anomaly (more frames causing more, not fewer, faults).

LRU (Least Recently Used) replaces the page whose most recent reference is furthest in the past, on the reasonable assumption that a page not used for a long time is unlikely to be used again soon.

Optimal replaces the page that will not be used for the longest time in the future — provably the fewest possible faults for any reference string and frame count — but requires knowledge of the future reference sequence, so it cannot be implemented in a real system; it exists only as a theoretical/benchmark lower bound used to judge how close a practical algorithm (like LRU or clock) comes to the best possible result.

Clock (second-chance) algorithm: an approximation of LRU that needs no exact timestamps; each frame has a single reference ("use") bit, set by hardware whenever the page is accessed; the replacement pointer sweeps frames in a circle, and for each frame it visits: if the use bit is 0, that frame is chosen as the victim; if it is 1, the bit is cleared to 0 and the pointer moves on (giving the page a "second chance" before it can be evicted on the pointer's next pass) — this gives most of LRU's benefit using only one bit per frame and O(1) amortised work per reference, instead of LRU's need for either a full access-time stamp or a doubly-linked usage list updated on every single reference.

Evaluation: Optimal is unbeatable in fault count but unimplementable in a live system, since it would need to know the future; it is used purely as the yardstick other algorithms are measured against. LRU never suffers Belady's anomaly because it has the stack property — the set of pages LRU would keep resident with k frames is always a subset of the set it would keep with k+1 frames, so adding frames can never force a page out that would otherwise have stayed, guaranteeing faults never increase as frames increase; FIFO lacks this stack property, which is exactly why it can exhibit Belady's anomaly. Hardware support LRU strictly needs is either a true access-time counter/stamp updated by hardware on every memory reference, or a hardware-maintained stack/linked list of pages in access order — both expensive in silicon and/or per-access overhead at scale; the clock algorithm avoids this by needing only one reference bit per frame, set cheaply by the MMU on access and cleared cheaply by the OS during the sweep, which is why clock (or variants such as the enhanced/clock-with-dirty-bit) is what real operating systems actually implement instead of exact LRU. For a workload with strong locality (as a web server's hot data commonly has), LRU and clock should both perform close to Optimal, since the "recently used" heuristic matches reality well; FIFO typically performs noticeably worse on such workloads because recency of insertion is a much weaker predictor of future use than recency of actual access.

Experimental investigation: generating reference strings with strong locality (repeating a small working set) versus uniformly random access and counting faults for each algorithm across 3–10 frames should show all three algorithms' fault counts falling as frames increase (monotonically for LRU/Optimal, possibly non-monotonically for FIFO due to Belady's anomaly), with the locality-based string showing a much steeper, earlier drop (faults fall close to zero once frames ≥ working-set size) than the random-access string, where

faults remain comparatively high even with many frames, since there is little pattern to exploit. Measuring running time for 100,000 references should show FIFO and the clock algorithm are both fast (O(1) amortised per reference), while an exact LRU implementation (as completed in the TODO below, scanning for the oldest last[] timestamp) and Optimal (scanning ahead in the reference string for the furthest-future use) are both materially slower per reference (O(frames) for LRU as implemented here, O(frames × remaining-string-length) for the straightforward Optimal implementation given), confirming why real systems use clock rather than exact LRU or Optimal in practice.

```
#include <stdio.h>
#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;

        /* TODO 1 solution */
        if (used < frames) {
            f[used] = ref[i];
            last[used] = i;
            used++;
        } else {
            int victim = 0;
            for (int j = 1; j < frames; j++)
                if (last[j] < last[victim]) victim = j;
            f[victim] = ref[i];
            last[victim] = i;
        }
    }
    return faults;
}

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len; /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
```

```
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
               lru(ref, len, fr), optimal(ref, len, fr));

    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

Why FIFO has more faults with 4 frames than with 3 for the second (Belady) reference string, and why LRU cannot behave like this: FIFO decides what to evict purely by insertion order, completely ignoring whether a page is still actively needed; adding a frame changes which page happens to be "oldest" at each future eviction point in a way that is not guaranteed to be monotonic, so by coincidence the 4-frame run can end up evicting a page that the 3-frame run had, by luck, kept resident — producing more faults overall even with strictly more memory available. LRU cannot exhibit this because of its stack property (noted above): the set of resident pages under LRU with k frames is always contained within the set resident with k+1 frames at every point in the reference string, so adding a frame can only ever keep a superset of what was kept before and can never cause an eviction that a smaller-frame run would have avoided — faults are therefore guaranteed to be monotonically non-increasing as frames increase for LRU (and for Optimal), but not for FIFO.

Alternative Concept and Complexity Analysis — the clock (second-chance) algorithm instead of exact LRU: time per reference falls from LRU's O(frames) work per fault (scanning all frames' last-use timestamps, as implemented in the TODO above) to clock's O(1) amortised work per reference (checking/clearing one bit and advancing a pointer; only pages whose bit was already 0 are evicted immediately, while repeated full sweeps are rare in practice for typical locality). Extra space per frame falls from a full integer timestamp (4–8 bytes) to a single bit (though in practice still rounded up to a byte or packed into existing page-table entry flag bits, since hardware already provides an access bit there at essentially no extra cost). In simulation, clock's page-fault count for workloads with reasonably strong locality typically comes very close to exact LRU's — often within a few percent — because the periodic sweep-and-clear approximates "not recently used" almost as well as a true timestamp, which is why operating systems accept the small fault- count gap in exchange for clock's much lower per-access bookkeeping cost.

Q6 – Disk Management (4 Marks)

Low-level formatting writes the physical sector structure (sector headers, ECC fields) onto the raw disk, usually done once by the manufacturer; partitioning divides the disk into one or more logical regions, each treated as an independent logical disk by the OS; logical formatting then creates a file system's own data structures (superblock, inode/metadata tables, free-space bitmap, root directory) within a partition so it can actually store files.

Boot block and boot process: a small, fixed region at a known location on the disk (the boot block/boot sector, or, on modern UEFI systems, the EFI System Partition) holds a minimal bootstrap loader; on power-on, firmware (BIOS/UEFI) reads and executes this bootstrap code, which in turn loads a fuller boot loader (e.g., GRUB) that finally loads the OS kernel into memory and transfers control to it — a short, carefully constrained chain because the bootstrap code itself must fit in a very small, fixed amount of space and run before any file-system driver is even available.

Bad blocks: the disk controller (or the OS/file system) maintains a bad-block list/table; a sector that fails is remapped transparently to a reserved spare sector (common on modern disks' own firmware) or, at the OS level, is marked in the file system's metadata as unusable and simply never allocated to a file, so higher software layers never need to deal with the failure directly.

Swap space: for a records server with predictable peak-load spikes, swap should be sized to comfortably cover the worst-case shortfall between the working set of active processes and physical RAM during a peak (commonly recommended, as a starting rule of thumb, as 1–2× RAM for moderate-RAM servers, though modern high-RAM database servers often deliberately keep swap small and instead rely on sufficient RAM,

treating heavy swapping itself as a performance problem to avoid rather than tolerate) and should be placed on the server's fastest available storage (a dedicated SSD partition, isolated from the database's own data volumes) so that, if swapping does occur, it does not compete for the same disk I/O bandwidth the database needs for ordinary reads/writes.

Evaluation — RAID levels: RAID 0 (striping, no redundancy) maximises capacity (100% of raw disk space usable) and speed but offers zero fault tolerance — any single disk failure loses all data, making it unsuitable alone for a records server. RAID 1 (mirroring) tolerates the loss of any one disk in each mirrored pair at the cost of using only half the raw capacity, with good read speed (reads can be served from either copy) but write speed limited to a single disk's speed (every write must go to both). RAID 5 (block-level striping with distributed parity) tolerates exactly one disk failure, uses (n–1)/n of raw capacity for n disks (more capacity-efficient than mirroring for larger arrays), but suffers a write penalty (every write requires reading and rewriting parity) and a risky, slow rebuild window after a failure, during which a second failure would cause data loss. RAID 10 (striped mirrors) combines RAID 1's fault tolerance with RAID 0's speed, tolerating at least one failure per mirrored pair (and sometimes more, depending on which specific disks fail) with strong read and write performance, at the cost of using only half the raw capacity, similar to plain mirroring. For the hospital records server, RAID 10 (or RAID 1 for a smaller array) is the recommended choice, because the records server's "must never stop" requirement (established in Assignment 1) favours RAID 10's strong fault tolerance and good write performance for an actively-updated database over RAID 5's better capacity efficiency but weaker write performance and riskier rebuilds.

Swap on a separate disk vs. a swap file: a dedicated swap partition/disk gives predictable, isolated I/O performance and avoids any file-system-layer overhead, which matters if swapping does occur under peak load; a swap file (a regular file used as swap space within an existing file system) is more flexible to resize without repartitioning but shares the same underlying disk's I/O bandwidth and file-system bookkeeping with everything else on that volume — for the records server, a dedicated swap partition on fast storage, separate from the database's own data volumes, is preferred to keep an occasional swap event from further slowing already-stressed database I/O.

Linux partition/swap inspection: lsblk lists block devices and their partitions in a tree view; fdisk -l (or parted -l) lists each disk's partition table in detail (start/end sectors, size, type); swapon --show or cat /proc/swaps lists active swap areas and their current usage.

Experimental investigation: showing the student's own system's partitions/swap with lsblk and swapon --show; calculating usable capacity of 6×4 TB disks: RAID 0 = 24 TB, RAID 1 = 12 TB (3 mirrored pairs, or as low as 4 TB if all 6 are one giant mirror set, depending on configuration — typically quoted as n/2 × disk size for simple mirroring), RAID 5 = (6–1)×4 = 20 TB, RAID 10 = (6/2)×4 = 12 TB; probability of data loss given a disk failure rate f (independent failures, simplified model): for RAID 1 (one mirrored pair), data loss requires both disks in the pair to fail before the first is replaced/rebuilt, roughly f × (rebuild- window failure probability of the second disk) — a small, second-order probability; for RAID 5 (n=6), data loss requires any second disk among the remaining 5 to fail during the rebuild window after the first failure, so the exposure is roughly 5× a single pair's exposure, i.e. a noticeably higher aggregate probability of a double failure during the (often longer, parity-rebuild) RAID 5 rebuild window compared to RAID 1's shorter, simpler mirror-resync window; measuring SSD vs hard-disk sequential read speed with dd or hdparm -t typically shows the SSD several times to over an order of magnitude faster, especially for random access.

Conclusion: the recommended storage configuration for the records server is RAID 10 across SSDs for the active database (prioritising availability and write performance for the "must never stop" requirement), with dedicated fast swap space isolated from the data volumes, and routine backups to separate, ideally off-site/replicated storage not counted as part of the RAID's own fault tolerance (RAID protects against disk failure, not against accidental deletion, corruption, or a site-level disaster).

Alternative Concept and Complexity Analysis — RAID 6 instead of RAID 5 (or RAID 10): usable capacity for 6×4 TB disks under RAID 6 (double distributed parity) is (6–2)×4 = 16 TB — less than RAID 5's 20 TB and also less than RAID 10's 12 TB would need (6 disks) to compare at... (note: RAID 10 here uses the same 6 disks for

12 TB, so RAID 6 gives more usable capacity than RAID 10 for the same disk count, but less than RAID 5). Time to write a block rises versus RAID 5, because RAID 6 must compute and write two independent parity blocks (two parity calculations) for every write instead of one, adding further write overhead on top of RAID 5's existing read-modify-write parity penalty. The key benefit is fault tolerance: RAID 6 survives any two simultaneous disk failures (versus RAID 5's one), which matters most for large arrays with long rebuild times, where the probability of a second failure occurring before a first failure's rebuild completes becomes significant — for the hospital's relatively modest 6-disk array, RAID 10 still offers comparable or better practical safety with better write performance, so RAID 6's main advantage (tolerating two failures) is most compelling for larger arrays than this one, making it a reasonable but not clearly superior alternative to the RAID 10 recommendation above for this specific 6-disk records server.

Q7 – Design a Memory-Management Plan from First Principles (10 Marks)

Stage 1 – Interpret the Problem (2 Marks) Memory needs and timing constraints: the real-time control task needs a small, bounded, predictable memory footprint and must never be delayed by a page fault (any fault-induced disk/flash I/O stall, even a few milliseconds, would violate its real-time guarantee, echoing the hospital monitor's 10 ms constraint from Assignment 1); the machine-learning task needs 1.5 GB of the device's 2 GB total RAM, is CPU/memory intensive but not latency-critical at the instruction level (an occasional page fault delays its result but causes no safety issue); the logging task needs a modest, slowly-growing amount of memory and tolerates ordinary demand-paging delays. Which tasks can tolerate page faults: the ML and logging tasks can; the real-time control task cannot. Measurable requirements: (i) the control task's memory must be 100% resident at all times (0 page faults ever, verified over a long soak test), (ii) the control task's worst-case loop delay must stay under its stated real-time bound (to be measured directly, per Stage 3), (iii) total resident memory for all tasks together must not exceed 2 GB with no swap device available, (iv) the ML task must have at least 1.5 GB reliably available to it without being starved by the other two tasks.

Stage 2 – Compare Candidate Mechanisms (3 Marks) Comparison table (summary): Contiguous allocation gives the most predictable, fault-free access (ideal for the control task) but is inflexible and wastes memory if sized generously — appropriate only for the small, fixed-size control task, not for the whole device. Plain paging (without locking) offers flexible, fine- grained allocation and no external fragmentation, but ordinary demand-paged pages can still be evicted and re-faulted at any time — unacceptable for the control task unless its pages are explicitly pinned (locked). Segmentation is not particularly suited to this problem (its main benefit, logical/protection grouping, is better achieved here simply by giving each task its own process/address space) and is omitted from the final design. Demand paging is the right default mechanism for the ML and logging tasks, since it lets their memory be managed flexibly within the 2 GB budget using ordinary page-fault handling, with no swap disk meaning faults can only be satisfied by reclaiming/discarding clean, re-loadable pages (e.g., re-readable program code) rather than by writing dirty pages out to a swap device that does not exist — so the device's design must ensure the ML/logging tasks' dirty working set still fits comfortably in RAM even without swap. Locking memory for the real-time task: Linux's mlock()/mlockall() system calls pin a range of a process's pages in physical RAM, preventing the kernel from ever paging them out or evicting them to service another process's fault, which directly satisfies requirement (i) above; mlockall(MCL_CURRENT | MCL_FUTURE) locks both the task's current pages and any pages it allocates in future, guarding against the task later growing its memory and inadvertently becoming unlocked. Decision: contiguous/locked memory (mlock-pinned pages) for the control task; ordinary demand-paged memory for the ML and logging tasks, sized so their combined resident footprint plus the control task's fixed footprint never exceeds 2 GB.

Stage 3 – Implement and Verify (3 Marks) A program for the time-critical loop calls mlockall(MCL_CURRENT | MCL_FUTURE) at start-up, then runs its control loop at a real-time scheduling priority (e.g., SCHED_FIFO via sched_setscheduler()), measuring the loop's iteration time with clock_gettime(CLOCK_MONOTONIC, ...) before and after each iteration and recording the worst-case (maximum) observed delay over a long run under background memory pressure from the other two tasks; the same measurement repeated with mlockall() removed should show a measurably worse (and far more variable) worst-case delay whenever the background tasks' memory pressure forces the control task's own pages to be evicted and re-faulted in. A simple

page-replacement simulator (reusing the FIFO/LRU/Optimal code from Q5) models the ML and logging tasks' combined memory use against the frames remaining after the control task's fixed, locked allocation, to confirm their combined working set is expected to fit without excessive faulting. Verifying the real-time task is never delayed: cross-checking /proc/<pid>/status for the control task shows VmLck (locked memory) equal to its full working set, and the measured worst-case loop delay (with locking enabled) stays within the task's required bound across repeated runs, including runs with the ML task actively consuming memory.

Stage 4 – Defend the Design (2 Marks) Justification from measurements: the locked-vs-unlocked timing comparison in Stage 3 is the direct evidence that mlockall() is necessary, not optional, for the control task to meet its deadline whenever the device is under real memory pressure from the ML task — exactly the scenario the vendor's roadmap (more tasks to come) makes likely rather than hypothetical. If a fourth task of 500 MB is added: the fixed 2 GB budget becomes tight (control task's locked allocation + 1.5 GB ML + logging + 500 MB new task could now exceed 2 GB), so either the ML task's allocation must be reduced/made adjustable, the new task must itself be carefully sized and monitored, or — since there is no swap disk to fall back on — physical RAM must be increased; this is analogous to Q4's demand-paging/EAT trade-off, except here a swap-based fallback is explicitly unavailable, making the memory budget a harder constraint than on a typical desktop/server system. One remaining limitation: locking the control task's memory permanently removes those frames from the pool available to every other task, even during periods when the control task is briefly idle, so the design trades some memory elasticity (the ML/logging tasks can never temporarily "borrow" the control task's reserved frames) for the hard real-time guarantee — an acceptable and necessary trade-off here, but one that must be re-evaluated (and the locked allocation re-sized) every time a new task is added to the device.

Alternative Concept and Complexity Analysis — a real-time operating system with no virtual memory instead of Linux with locked memory: worst-case delay for the control task improves further under a true RTOS with no virtual memory at all, since there is no page-fault mechanism whatsoever to interact with (no MMU-based demand paging to disable/avoid), removing an entire class of possible jitter sources that even mlock()-based locking on a general-purpose OS cannot fully eliminate (e.g., very rare kernel-internal latency spikes unrelated to the locked pages themselves); a dedicated RTOS can typically offer a tighter, more easily-certified worst-case bound. Memory available to the machine-learning task falls, however, because an RTOS without virtual memory generally has no demand-paging or memory-overcommit mechanism at all — every task's memory must be statically or simply allocated up front from the same fixed 2 GB pool, removing Linux's flexibility to let the ML and logging tasks' resident footprints flex somewhat day to day within the remaining budget. Effort of adding new tasks rises on a no-virtual-memory RTOS, since each new task's exact memory budget must typically be planned and reserved explicitly in advance (no OS-level demand paging to absorb moderate, unplanned growth), compared with Linux, where the vendor can add a modest new task and rely on ordinary demand paging to accommodate it, provided it is not also real-time-critical.